For instance, to refine the example summing numbers in a file, we might decide that we skip odd numbers and we use `0` as a marker, that means we stop summing numbers as soon as we read a `0`.

```
while not eof(file) do
  x ← read(file)
  if x % 2 == 1 then
    continue
  elsif x == 0 then
    break
  total ← x
```

Whenever the next number read from the file is odd, then we skip to the next iteration of the loop (meaning we will check again the predicate, and so if we are at the end of the file).

If, instead, the number is `0`, we will exit the loop before incrementing our total.

Both keywords can also be used inside `for` loops.

A.5 Blocks and Indent

So far, in most of our examples, loops and each branch of our conditionals consisted of exactly one instruction, and this made syntax particularly simple. However, that's not the case in general, and each branch of the `if-then-else` statement can execute an arbitrary number of instructions.